

Heart Valve Disease Classification from Phonocardiogram Signals using Transfer Learning Techniques

*Report submitted to the SASTRA Deemed to be University
as the requirement for the course*

ICT300 - MINI PROJECT

Submitted by

CHRISTY EVANJALINE J

(Reg. No.: 127014016, B. Tech Information and Communication Technology)

MOHAMED RIZVI ABDULLA HADHI

(Reg. No.: 127014032, B. Tech Information and Communication Technology)

MOHAN RAJ R.V

(Reg. No.: 127014034, B. Tech Information and Communication Technology)

May 2026



SCHOOL OF COMPUTING
THANJAVUR, TAMIL NADU, INDIA – 613 401



**SCHOOL OF COMPUTING
THANJAVUR – 613 401**

BONAFIDE CERTIFICATE

This is to certify that the report titled “**Heart Valve Disease Classification From Phonocardiogram Signals using Transfer Learning Techniques**” submitted as a requirement for the course, ICT300: MINI PROJECT for B.Tech. is a Bonafide record of the work done by **Miss. CHRISTY EVANJALINE J** (Reg. No. 127014016, B. Tech Information and Communication Technology), **Mr. MOHAMED RIZVI ABULLA HADHI** (Reg. No.: 127014032, B.Tech Information and Communication Technology) and **Mr. MOHAN RAJ R.V.** (Reg. No.: 127014034, B. Tech Information and Communication Technology) during the academic year 2025-26, in the School of Computing, under my supervision.

Signature of Project Supervisor :

Name with Affiliation : Dr B. Lakshmi , AP/SOC

Date :

Mini Project *Viva voce* held on _____

Examiner 1

Examiner2

ACKNOWLEDGEMENTS

We would like to thank our Honourable Chancellor **Prof. R. Sethuraman** for providing us with an opportunity and the necessary infrastructure for carrying out this project as a part of our curriculum.

We would like to thank our Honourable Vice-Chancellor **Dr. S. Vaidhyasubramaniam** and **Dr. S. Swaminathan**, Dean, Planning & Development, for the encouragement and strategic support at every step of our college life.

We extend our sincere thanks to **Dr. R. Chandramouli**, Registrar, SASTRA Deemed to be University for providing the opportunity to pursue this project.

We extend our heartfelt thanks to **Dr. V. S. Shankar Sriram**, Dean, School of Computing, **Dr. R. Muthaiah**, Associate Dean, Research, **Dr. K. Ramkumar**, Associate Dean, Academics, **Dr. D. Manivannan**, Associate Dean, Infrastructure, **Dr. R. Alageswaran**, Associate Dean, Students Welfare for the successful completion of the project

Our guide **Dr. Lakshmi. B**, Associate Professor-II, School of Computing, was the driving force behind this whole idea from the start. Her deep insight in the field and invaluable suggestions helped us in making progress throughout our project work. We also thank the project review panel members for their valuable comments and insights which made this project better.

We would like to extend our gratitude to all the teaching and non-teaching faculties of the School of Computing who have either directly or indirectly helped us in the completion of the project.

We gratefully acknowledge all the contributions and encouragement from my family and friends resulting in the successful completion of this project. We thank you all for giving us an opportunity to showcase our skills through project.

ABSTRACT

Heart valve diseases are common cardiovascular disorders that require early and accurate diagnosis to avoid severe health complications. Conditions such as Aortic Stenosis (AS), Mitral Regurgitation (MR), Mitral Stenosis (MS), and Mitral Valve Prolapse (MVP) need proper detection for timely treatment. Conventional diagnostic methods like echocardiography, Magnetic Resonance Imaging, and Cardiac Computed Tomography provide accurate results but are expensive and require advanced hospital facilities. Phonocardiogram (PCG) signals offer a low cost and non-invasive alternative for analysing heart sounds, but their interpretation depends mainly on physician expertise. This project focuses on developing an automatic and efficient system for classifying heart valve diseases using PCG signals with machine learning and Transfer Learning (TL) techniques to reduce dependency on manual diagnosis. Two public datasets are used in this work, namely the Yaseen dataset containing five classes including Normal, Aortic Stenosis, Mitral Regurgitation, Mitral Stenosis, and Mitral Valve Prolapse, and the PhysioNet 2016 dataset for validation. The signals are pre-processed using resampling, band pass filtering, and normalization to improve signal quality and consistency. Three time-frequency representations, namely spectrogram, log-Mel spectrogram, and scalogram, are compared, where spectrogram provides better representation by effectively capturing both time and frequency characteristics of heart sounds. Transfer learning is implemented using the lightweight pre-trained YAMNet model, and its performance is compared with MobileNet and other convolutional neural network models. The results show that the spectrogram-based YAMNet model performs efficient heart valve disease classification with good computational efficiency and robustness to noise, making it suitable for practical healthcare applications.

KEYWORDS: Heart Valve Disease, PCG Signals, Transfer Learning, Spectrogram Analysis, YAMNet, Heart Sound Analysis.

Table of Contents

Title	Page No.
Bonafide Certificate	ii
Acknowledgements	iii
Abstract	v
List of Tables	vi
List of Figures	vi
Abbreviations	viii
1. Introduction	1
1.1 Base Paper	1
1.2 Related Work	1
1.3 Problem Statement	2
1.4 Objective	2
1.5 Merits	4
1.6 Demerits	5
2. Problem Definition and Methodology	6
2.1 Proposed Solution	6
2.2 Block Diagram	7
3. Results and Discussion	8
3.1 Data Preprocessing	8
3.2 Model Modification	9
3.3 Results of PhysioNet Dataset	10
3.4 Performance Metrics	11
3.5 Performance Metrics Comparison Between Base Paper and Our Work	12
4. Conclusion and Future Work	13
4.1 Conclusion	13
4.2 Future Plans	14

5. References	15
6. Appendices	16
6.1 Appendix A: Sample Source Code	16
6.2 Appendix B: Screenshots	21

List of Tables

Table No	Title	Page No
3.5.1	Accuracy for Yaseen Dataset	12
3.5.2	PhysioNet Dataset Evaluation	12

List of Figures

Figure No	Title	Page No
1.4.1	Noisy and Pre-processed PCG Signal	3
1.4.2	Time-Frequency Representation of PCG Signal for Normal and Abnormal Cases	3
2.3.1	Block Diagram of Proposed Method	7
3.1.1	Fragment Generation	8

3.2.1	Replacing the 512 classes to 5 classes	9
3.3.1	Model Performance Evaluation on PhysioNet Dataset Using Linear Spectrogram	10
3.4.1	Model Performance Evaluation for Three types of Frequency Representation	11
3.4.2	Accuracy Comparison at Fragment and Subject Levels Across Trainable Layer	11
6.2.1	Model Performance Evaluation on Yassen Dataset Using Linear Spectrogram signal	21
6.2.2	Model Performance Evaluation on Yassen Dataset Using Scalogram signal	21
6.2.3	Model Performance Evaluation on Yassen Dataset Using Log-mel Spectrogram signal	21
6.2.4	Model Performance Evaluation on PhysioNet Dataset Using Linear Spectrogram	22
6.2.5	Final layer of Modified YAMNet	22
6.2.6	Fragment Extracted	22
6.2.7	Prediction made by Model trained with Yaseen Dataset (Spectrogram)	22

ABBREVIATIONS

PCG – Phonocardiogram

HVD – Heart Valve Disease / Disorder

TL – Transfer Learning

YAMNet – Yet Another Mobile Network

TF – Time-Frequency

AS – Aortic Stenosis

MR – Mitral Regurgitation

MS – Mitral Stenosis

MVP – Mitral Valve Prolapse

CNN – Convolutional Neural Network

STFT – Short-Time Fourier Transform

CWT – Continuous Wavelet Transform

SVM – Support Vector Machine

DL – Deep Learning

MFCC – Mel-Frequency Cepstral Coefficient

SNR – Signal-to-Noise Ratio

CHAPTER 1

INTRODUCTION

1.1 Base Paper

Heart valve diseases are a major cause of cardiovascular complications worldwide and require early and accurate diagnosis to prevent severe health risks. Conditions such as Aortic Stenosis (AS), Mitral Regurgitation (MR), Mitral Stenosis (MS), and Mitral Valve Prolapse (MVP) affect the normal functioning of the heart and can become life-threatening if not detected early. Conventional diagnostic techniques, including echocardiography, magnetic resonance imaging (MRI), and cardiac computed tomography (CT), provide reliable results but are expensive and depend on advanced medical infrastructure, limiting their accessibility in resource-constrained and rural areas.

Phonocardiogram (PCG) signals offer a low-cost and non-invasive alternative for analysing heart sounds, but their interpretation is highly dependent on medical expertise and can be subjective. To address this, recent advancements in Machine Learning (ML) and Deep Learning (DL) enable automated analysis of biomedical signals. In particular, Transfer learning using pre-trained models such as YAMNet, along with time-frequency representations like spectrograms, log-Mel spectrograms, and scalograms, allows effective extraction of both temporal and frequency features. This project proposes an automated system for classifying heart valve diseases using spectrogram-based features and transfer learning, aiming to improve accuracy while maintaining low computational complexity. The system is designed to be robust to noise and suitable for real-time deployment, making it a practical solution for accessible and early diagnosis.

1.2 Related Work

Several studies have explored PCG signal classification using signal processing and deep learning techniques.

Yaseen et al. [4] proposed a multi-feature-based classification approach using statistical and frequency-domain features, showing improved performance on smaller datasets but relying on manual feature extraction.

Clifford et al. [18] introduced the PhysioNet Challenge dataset, a benchmark dataset with noisy real-world PCG recordings used for evaluating classification models.

Zeng et al. [30] utilized advanced time–frequency techniques such as TQWT and FA-MVEMD for heart valve disease detection, providing rich feature representation at the cost of higher computational complexity.

Bozkurt et al. [54] demonstrated the effectiveness of spectrogram and log-Mel representations for CNN-based heart sound classification, highlighting the importance of time–frequency features.

Koike et al. [56] explored transfer learning for PCG classification, showing that pretrained audio models improve performance with limited data.

Plakal and Ellis [59] developed YAMNet, a pretrained MobileNet based audio classification model trained on Audio Set, widely used for transfer learning. **Kautz et al. [61]** introduced evaluation metrics for multi-class classification, essential for performance analysis in medical diagnosis systems.

1.3 Problem Statement

Heart valve diseases are commonly diagnosed using imaging techniques such as echocardiography, Magnetic Resonance Imaging (MRI), and Cardiac Computed Tomography (CT), which provide accurate results but are expensive, time-consuming, and not easily accessible in rural or low-resource settings. The proposed system focuses on developing an automatic, low-cost, and efficient solution for diagnosing heart valve diseases by classifying heart sounds using deep learning techniques.

1.4 Objective

- To investigate the effectiveness of **Transfer Learning using pre-trained convolutional neural network** for PCG classification.
- Make a comparative study between 2 different datasets, **Yaseen et. al dataset** and **Physio-net challenge 2016 dataset**
- To extract time-frequency representations of heart sound signals such as **spectrogram**, log-mel spectrogram, scalogram for feature analysis.

- To fine-tune lightweight pre-trained models for accurate classification of heart valve diseases.
- To investigate the computational efficiency and robustness against practical **noise** contamination for performance evaluation in a possible real-life scenario.

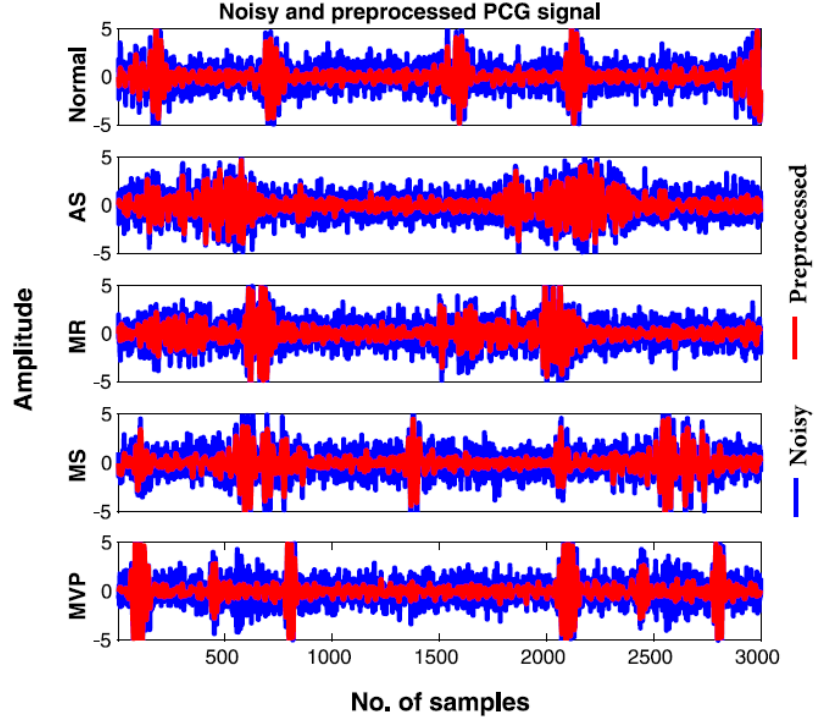


Fig 1.4.1 Noisy and Pre-processed PCG Signal

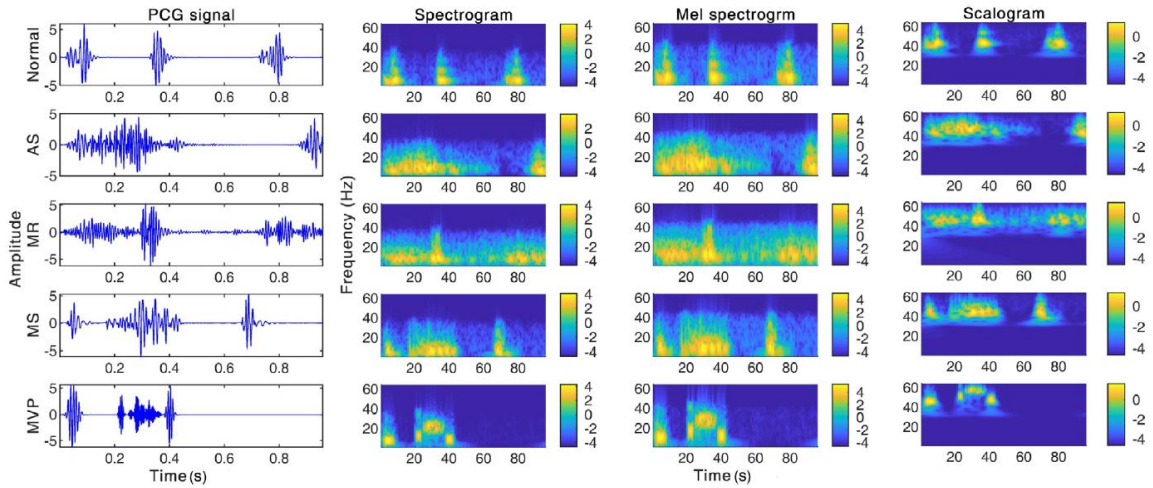


Fig 1.4.2 Time-Frequency Representation of PCG Signal for Normal and Abnormal

1.5 Merits

High Classification Accuracy:

The base paper achieved excellent classification performance using the YAMNet-based transfer learning model on PCG signals. The model obtained an overall accuracy of **99.83%**, sensitivity of **99.59%**, and specificity of **99.90%** on the Yaseen dataset for five class heart valve disease classification. For binary classification on the PhysioNet 2016 dataset, the model achieved **92.23% accuracy**, showing strong performance even on a more challenging dataset.

Effective Transfer Learning Approach:

Instead of training a deep neural network from scratch, the proposed method utilizes the lightweight pre-trained **YAMNet** model based on MobileNet architecture. This reduces training complexity, requires less training data, and improves feature extraction performance for PCG signal classification.

Robust Time Frequency Feature Representation:

The study compares Spectrogram, Log-Mel Spectrogram, and Scalogram representations. Among them, Spectrogram provided better performance by effectively capturing both time and frequency characteristics of heart sounds, resulting in improved disease classification accuracy.

Computational Efficiency:

YAMNet is a lightweight model with lower memory requirements and faster execution compared to large CNN models. It reduces preprocessing complexity, making heart sound classification computationally efficient for healthcare applications.

Noise Robustness:

The proposed method was evaluated under noisy conditions using breathing noise, hospital noise, and Gaussian noise. The model maintained good classification performance, showing robustness in real world noisy recording environments.

1.6 Demerits

Dependence on Data Quality:

Although the model performs well, its accuracy depends heavily on the quality of PCG recordings. Poor quality recordings, excessive noise, or missing heart sound components may affect classification performance.

Limited Dataset Availability:

The study uses only publicly available datasets such as Yaseen and PhysioNet 2016. The availability of large-scale labelled heart valve disease datasets is limited, which restricts model generalization across diverse patient populations.

Class Imbalance Challenges in Practical Data:

Although the Yaseen dataset is balanced, real world hospital datasets are often imbalanced. This may reduce model performance when applied to actual clinical scenarios.

Hardware Integration Challenges:

Although the proposed model is lightweight, implementing it on hardware devices can still be challenging due to compatibility issues, sensor integration requirements, and the need for proper real-time system configuration. This may increase deployment complexity in practical healthcare environments.

CHAPTER 2

PROBLEM DEFINITION AND METHODOLOGY

2.1 Proposed Solution

The proposed system follows a structured pipeline that transforms raw PCG signals into accurate classification outputs.

Initially, PCG signals are acquired from publicly available datasets such as PhysioNet and Yaseen. Since these signals may contain noise and inconsistencies, preprocessing is performed to improve data quality. All signals are resampled to a uniform frequency of **2000 Hz**, followed by the application of a **band-pass filter (25–600 Hz)** to remove unwanted noise while preserving important heart sound components. The signals are then normalized using **Z-score normalization** to standardize amplitude variations.

After preprocessing, the one-dimensional PCG signals are converted into two-dimensional time–frequency representations. This step is essential because Convolutional Neural Networks perform more effectively on image-like inputs. Three types of representations are used:

- Spectrogram (Short-Time Fourier Transform)
- Log-Mel Spectrogram
- Scalogram (Continuous Wavelet Transform)

These representations capture both temporal and frequency characteristics of heart sounds, improving feature learning.

To ensure sufficient diagnostic information, the signals are segmented into overlapping fragments of **1.5 seconds**, allowing each segment to capture at least one complete cardiac cycle. Overlapping helps preserve continuity and reduces the risk of losing critical features. Each segment is resized to **96 × 64** dimensions for compatibility with the model.

The processed inputs are then fed into the modified YAMNet model for classification. The system is trained and evaluated using separate datasets, and performance is measured using accuracy and classification metrics.

By integrating signal preprocessing, advanced feature extraction, and transfer learning, the proposed solution provides an efficient, scalable, and accurate approach for automated heart valve disease classification.

2.2 Block Diagram

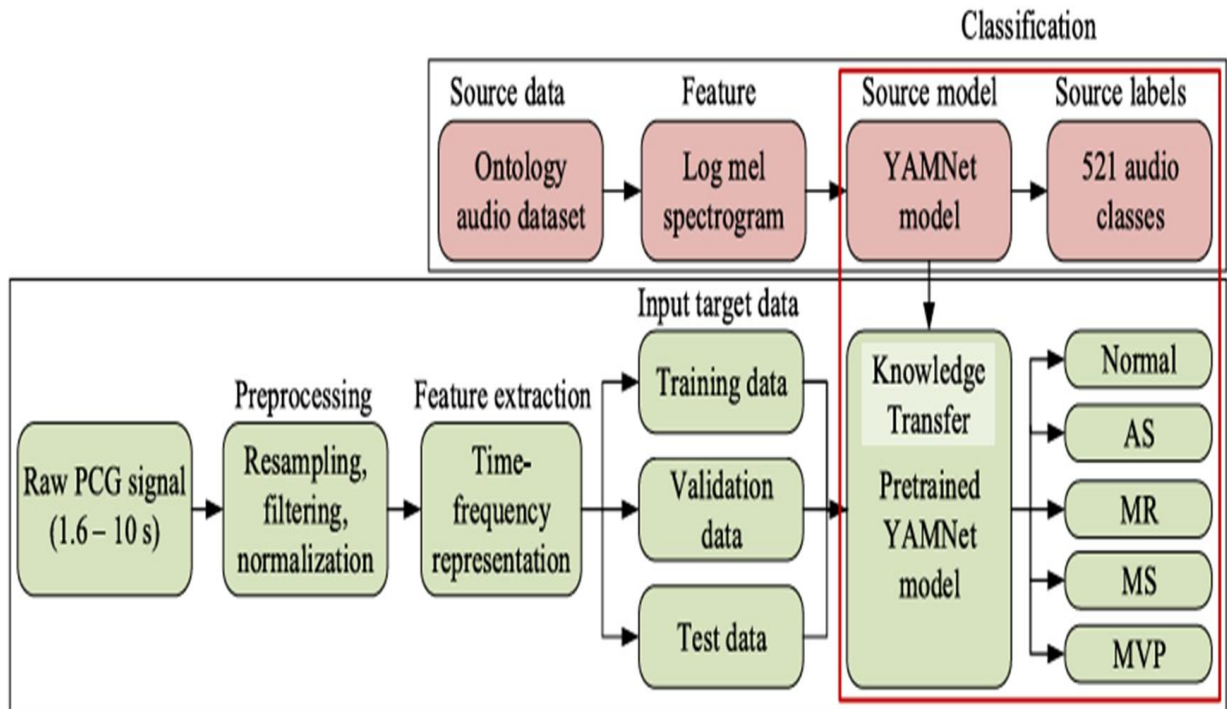


Fig 2.3.1 Block Diagram of Proposed Method

CHAPTER 3

RESULTS AND DISCUSSION

3.1 Data Preprocessing

The PCG (phonocardiogram) signals are first resampled to a uniform sampling rate to ensure consistency across all recordings. A band-pass filter is applied to remove noise and retain only the relevant heart sound frequencies. The signals are then normalized using Z-score normalization to standardize the amplitude distribution. After this, Short-Time Fourier Transform (STFT) is used to convert the signal into a spectrogram representation, followed by logarithmic scaling to enhance important features. The spectrogram is divided into overlapping fragments (150 frames with 50% overlap), resized to a fixed size (96×64), and normalized before being fed into the model.

```
def create_spectrogram_fragments(signal):  
  
    spec = np.abs(librosa.stft(  
        signal,  
        n_fft=128,  
        win_length=50,  
        hop_length=20  
    )) # compute spectrogram (STFT)  
  
    spec = np.log1p(spec) # log scaling  
  
    fragment_frames = 150 # frames per fragment  
    overlap = 0.5 # 50% overlap  
  
    step = int(fragment_frames * (1 - overlap)) # step size  
  
    fragments = [] # store fragments  
  
    for start in range(0, spec.shape[1] - fragment_frames + 1, step):  
  
        frag = spec[:, start:start + fragment_frames] # slice fragment  
  
        frag = tf.image.resize(  
            frag[...], np.newaxis,  
            (96, 64)  
        ).numpy() # resize for model input  
  
        frag = frag / np.max(frag) # normalize fragment  
  
        fragments.append(frag) # add to list  
  
    return fragments # return all fragments
```

Fig 3.1.1: Fragment Generation

3.2 Model Modification

To effectively perform classification, a pretrained deep learning model is utilized and adapted for the specific medical task.

The base model used is **YAMNet**, which is originally designed for general audio classification and trained on a large-scale dataset. Although powerful, the original architecture is not directly suitable for heart sound classification.

```
net = yamnet;
numLabels = 5;
inputSize = 1024;
layers = net.Layers;
lgraph = layerGraph(layers);
weights = randn([numLabels inputSize]) * 0.01;
bias = zeros([numLabels 1]);
lgraph = replaceLayer(lgraph, "dense", ...
    fullyConnectedLayer(numLabels, ...
        "Name","dense", ...
        "Weights",weights, ...
        "Bias",bias));

classNames = ["class1","class2","class3","class4","class5"];

lgraph = replaceLayer(lgraph, "Sound", ...
    classificationLayer("Name","Sounds", ...
        "Classes",classNames));

analyzeNetwork(lgraph);
net_final = assembleNetwork(lgraph);
exportPath = "C:\Users\mohan\Desktop\yassen_final_tuned";
exportNetworkToTensorFlow(net_final, exportPath);

disp("TensorFlow export completed successfully ✅");
```

Fig 3.2.1 Replacing the 512 classes to 5 classes

Therefore, the following modifications are carried out:

- The original fully connected layer corresponding to **521 audio classes** is removed, as it is not relevant to PCG-based classification.

- A new dense layer is introduced to enable task-specific classification. The modified model supports:
 - **Binary classification** (Normal vs Abnormal) for the PhysioNet dataset
 - **Multi-class classification** (Normal, AS, MR, MS, MVP) for the Yaseen dataset
- The internal log-Mel spectrogram extraction block of YAMNet is bypassed. This allows the model to directly accept externally generated time–frequency representations, giving better control over feature extraction.
- The modified architecture is implemented and trained using the **TensorFlow framework**, with the Adam optimizer used for efficient convergence.
- These modifications ensure that the pretrained model is effectively adapted to the domain of heart sound analysis.

3.3 Results of PhysioNet Dataset



```

print(f"Overall Accuracy: {round(accuracy * 100)}%")
print("Sensitivity:", sensitivity)
print("Specificity:", specificity)
print("Precision:", precision)
print("F1 Score:", f1)
print("SNR:", snr)
print("MCC:", mcc)
print("Kappa:", kappa)

... Overall Accuracy: 91%
Sensitivity: 0.8529431030234219
Specificity: 0.8529431030234219
Precision: 0.8570981485377656
F1 Score: 0.8550155778312503
SNR: 9.445405613222372
MCC: 0.710029094133016
Kappa: 0.7099945994263732

from sklearn.metrics import recall_score

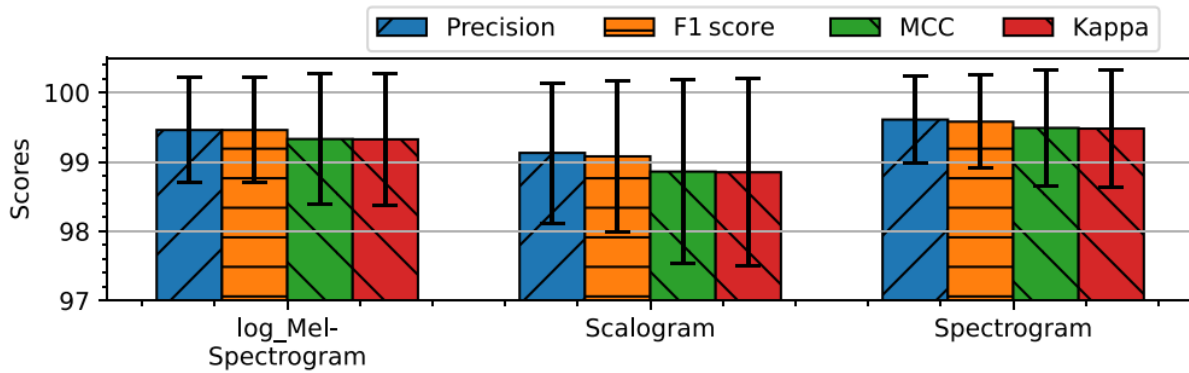
uar = recall_score(y_val, y_pred, average='macro')
print(f"Overall Accuracy: {round(accuracy * 100)}%")
print(f"UAR: {uar*100:.2f}")

Overall Accuracy: 91%
UAR: 85.29

```

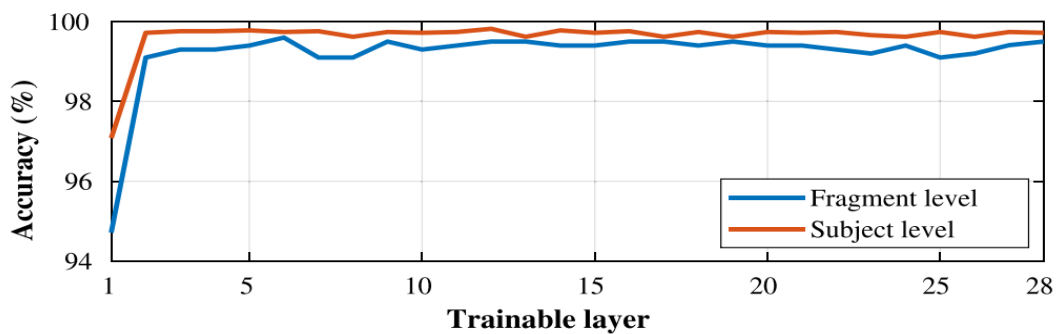
Fig 3.3.1: Model Performance Evaluation on PhysioNet Dataset Using Linear Spectrogram

3.4: Performance Metrics



The proposed model was evaluated using different audio feature representations, including Log-Mel Spectrogram, Scalogram, and Spectrogram. Performance metrics such as Precision, F1-score, MCC, and Kappa showed consistently high results across all feature types, demonstrating the robustness and reliability of the heart sound classification framework. Among them, the Spectrogram feature achieved comparatively better classification performance.

Fig 3.4.1: Model Performance Evaluation on Three type of Frequency Representation



The impact of varying trainable layers on model accuracy was analyzed at both fragment and subject levels. A significant improvement in accuracy was observed in the initial trainable layers, followed by stable performance across deeper layers. Subject-level accuracy remained consistently higher than fragment-level accuracy throughout the experiment. These findings indicate that fine-tuning deeper layers enhances model stability and overall classification performance.

Fig 3.4.2: Accuracy Comparison at Fragment and Subject Levels Across Trainable Layer

3.5 Performance Metrics Comparison Between Base Paper and Our Work

3.5.1 Accuracy for Yaseen Dataset

FREQUENCY REPRESENTATION	BASE PAPER	OUR WORK
LINEAR SPECTROGRAM	99.83%	96%
LOG-MEL SPECTROGRAM	99.79%	95%
SCALOGRAM	99.63%	93%

The accuracy of different frequency representations was analyzed on the Yaseen dataset. Among the evaluated methods, the Linear Spectrogram achieved the highest accuracy of 96%, while the Log-Mel Spectrogram and Scalogram obtained accuracies of 95% and 93%, respectively. These results indicate that frequency representation plays a crucial role in improving heart sound classification performance.

3.5.2 PhysioNet Dataset Evaluation (Spectrogram Representation)

PARAMETER	BASE PAPER	OUR WORK
ACCURACY	92.23%	91%
UAR	85.34%	85.25%

The model performance was further validated on the PhysioNet dataset using Spectrogram representation. Our implementation achieved an accuracy of 91% and a UAR of 85.25%, which are closely aligned with the base paper results, demonstrating consistent classification performance.

CHAPTER 4

CONCLUSION AND FUTURE PLANS

4.1 Conclusion

This study successfully demonstrated the effectiveness of transfer learning techniques for heart valve disease classification using Phonocardiogram (PCG) signals. The proposed model utilized signal preprocessing, time-frequency feature extraction, and a modified YAMNet architecture for automated heart valve disease classification. Different feature representations such as spectrogram, log-Mel spectrogram, and scalogram were analysed, where the spectrogram-based approach achieved better classification performance by effectively capturing both temporal and frequency characteristics of heart sounds. The model was evaluated using the Yaseen and PhysioNet datasets, where the Yaseen dataset provided higher accuracy under ideal noise-free conditions, while the PhysioNet dataset introduced realistic hospital noise, making the classification task more challenging. The obtained **91% accuracy** on the PhysioNet dataset indicates strong robustness and practical applicability under real-world noisy conditions. In addition, the use of transfer learning reduced training complexity and improved computational efficiency compared to training deep learning models from scratch.

The findings highlight,

- **Effective Feature Representation:** Spectrogram-based input provided better classification performance compared to log-Mel spectrogram and scalogram representations for heart sound analysis.
- **Computational Efficiency:** The lightweight YAMNet model reduced training complexity and computational cost while maintaining strong classification accuracy.
- **Real-World Applicability:** The model demonstrated reliable performance under noisy recording conditions, showing strong potential for practical healthcare applications and early heart valve disease screening.

4.2 Future Plans

The proposed heart valve disease classification system can be further improved in several ways in future. Even though the current model using transfer learning with YAMNet provides high classification accuracy, advanced deep learning models such as CNN-LSTM, attention-based models, or hybrid architectures can be explored to improve feature learning and classification performance. Combining multiple pretrained models may also help improve robustness and reduce misclassification.

Another important future enhancement is the use of larger and more diverse datasets. The current work mainly uses publicly available datasets such as Yaseen and PhysioNet 2016. In future, real-time heart sound recordings collected from hospitals, digital stethoscopes, and wearable devices can be included to improve model generalization and practical reliability. Additional heart sound abnormalities and rare valve disorders can also be incorporated for broader disease detection.

The proposed model can also be deployed as a web application or mobile-based system for real-time heart sound classification. Users can upload PCG recordings or record heart sounds directly, and the system can provide instant prediction results through a simple interface. This will be useful for early diagnosis, especially in rural areas and low-resource healthcare settings.

Future work can also focus on real-time audio processing, low-latency prediction, and secure handling of medical data for telemedicine applications. Integration with hospital systems and remote healthcare platforms can further improve accessibility and support continuous patient monitoring. Improving model performance under real-world noisy environments is also an important area for future research.

CHAPTER 5

REFERENCES

- [1] G.-Y. Yaseen, S. Son, and S. Kwon, Classification of heart sound signals using multiple features
- [2] G. D. Clifford, C. Liu, B. Moody, D. Springer, I. Silva, Q. Li, and R. G. Mark, Classification of normal/abnormal heart sound recordings: The PhysioNet/Computing in Cardiology Challenge 2016, Computing in Cardiology, 2016.
- [3] W. Zeng, Z. Lin, C. Yuan, Q. Wang, F. Liu, and Y. Wang, Detection of heart valve disorders from PCG signals using TQWT and FA-MVEMD.
- [4] B. Bozkurt, I. Germanakis, and Y. Stylianou, A study of time-frequency features for CNN-based automatic heart sound classification, IEEE Transactions on Biomedical Engineering.
- [5] T. Koike, K. Qian, Q. Kong, M. D. Plumbley, B. W. Schuller, and Y. Yamamoto, Audio for audio is better? Transfer learning models for heart sound classification.
- [6] M. Plakal and D. Ellis, YAMNet: Pre-trained audio event classification model, 2021.
- [7] J. F. Gemmeke, D. P. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, Audio Set: An ontology and human-labeled dataset for audio events, IEEE ICASSP, 2017

CHAPTER 6

APPENDICES

6.1 Appendix A: Sample Source Code

Audio Bandpass Filtering

```
from scipy.signal import butter, filtfilt
def bandpass_filter(signal, sr, low=25, high=600, order=10):
    nyquist = 0.5 * sr
    low = low / nyquist
    high = high / nyquist
    b, a = butter(order, [low, high], btype='band')
    filtered = filtfilt(b, a, signal)
    return filtered
```

Audio Signal Preprocessing and Normalization

```
def preprocess_signal(path):
    try:
        signal, sr = librosa.load(path, sr=None)
    except Exception as e:
        print("Error loading:", path)
        print(e)
        return None

    print("Loaded:", path)

    if sr != 2000:
        signal = librosa.resample(signal, orig_sr=sr,
target_sr=2000)
        sr = 2000

    signal = bandpass_filter(signal, sr)

    std = np.std(signal)

    if std == 0:
        print("STD ZERO:", path)
        return None

    signal = (signal - np.mean(signal)) / std
```



```

duration = len(signal) / sr
print("Duration:", duration)

if duration < 1.6:
    print("Too short:", path)
    return None

if duration > 10:
    signal = signal[:10 * sr]

return signal.astype(np.float32)

```

Representating the signal in Log-Mel Spectrogram

```

def create_logmel_fragments(signal, sr=2000):

    mel_spec = librosa.feature.melspectrogram(
        y=signal,
        sr=sr,
        n_fft=256,
        hop_length=64,
        n_mels=64
    )

    log_mel = librosa.power_to_db(mel_spec, ref=np.max)

    fragment_frames = 150
    hop = 4

    fragments = []

    for start in range(0, log_mel.shape[1], hop):

        frag = log_mel[:, start:start + fragment_frames]

        if frag.shape[1] < fragment_frames:
            pad_width = fragment_frames - frag.shape[1]
            frag = np.pad(frag, ((0,0),(0,pad_width)),
mode='constant')

        frag = tf.image.resize(
            frag[..., np.newaxis],
            (96, 64)
        ).numpy()

        frag = (frag - np.min(frag)) / (np.max(frag) - np.min(frag)
+ 1e-8)

```

```

        fragments.append(frag)

    return fragments

```

Representating the signal in Linear Spectrogram

```

def create_linear_fragments(signal):

    spec = np.abs(librosa.stft(
        signal,
        n_fft=128,
        win_length=50,
        hop_length=20
    ))

    spec = np.log1p(spec)

    fragment_frames = 150
    fragments = []

    for start in range(0, spec.shape[1], fragment_frames):

        frag = spec[:, start:start + fragment_frames]

        if frag.shape[1] < fragment_frames:
            pad_width = fragment_frames - frag.shape[1]
            frag = np.pad(frag, ((0,0),(0,pad_width)),
mode='constant')

        frag = tf.image.resize(
            frag[..., np.newaxis],
            (96, 64)
        ).numpy()

        frag = frag / np.max(frag)

        fragments.append(frag)

    return fragments

```

Representating the signal in Scalogram Spectrogram

```

def create_scalogram_fragments(signal, sr=2000):

    scales = np.arange(1, 64)

    coef, _ = pywt.cwt(signal, scales, 'morl')

```

```

scalogram = np.abs(coef)

scalogram = scalogram / np.max(scalogram)
scalogram = np.log(scalogram + 1e-6)

fragment_frames = 150
overlap = 0.5
step = int(fragment_frames * (1 - overlap))

fragments = []

for start in range(0, scalogram.shape[1] - fragment_frames + 1,
step):

    frag = scalogram[:, start:start + fragment_frames]

    frag = tf.image.resize(
        frag[..., np.newaxis],
        (96, 64)
    ).numpy()

    fragments.append(frag)

return fragments

```

Layer Freezing and Fine-Tuning in Neural Network

```

freeze = False

for layer in net.layers:

    if layer.name == "conv2d_12_":
        freeze = True

    if freeze:
        layer.trainable = False

    if layer.name == "fc_new_":
        layer.trainable = True
        freeze = False

```

Dataset Loading and Fragment Extraction Pipeline

```
import os
import numpy as np

def load_dataset(dataset_path):

    X = []
    y = []

    class_names = ["N_New", "MR_New", "AS_New", "MS_New", "MVP_New"]

    for label, class_name in enumerate(class_names):

        folder = os.path.join(dataset_path, class_name)

        for file in os.listdir(folder):

            if file.endswith(".wav"):

                path = os.path.join(folder, file)

                signal = preprocess_signal(path)

                if signal is None:
                    continue

                fragments = create_linear_fragments(signal)

                for frag in fragments:
                    X.append(frag)
                    y.append(label)

                if len(X) % 200 == 0:
                    print("Processed:", len(X))

    return np.array(X), np.array(y)
```

6.2 Appendix B: Screenshots

```
Overall Accuracy: 96%  
Sensitivity: 0.9584773914912128  
Specificity: 0.9896200931642921  
Precision: 0.9603029110936271  
F1 Score: 0.95938928289654  
SNR: 172000000029.18  
MCC: 0.9485140747735151  
Kappa: 0.9481488364683862
```

```
... Overall Accuracy: 96%  
UAR: 95.85
```

Fig 6.2.1: Model Performance Evaluation on Yassen Dataset Using Linear Spectrogram signal

```
... Accuracy: 93.11517563327466  
Precision: 0.9239045353149739  
UAR: 0.929078195057716  
F1 Score: 0.9219112118134066
```

Fig 6.2.2: Model Performance Evaluation on Yassen Dataset Using Scalogram signal

```
▼ ... Overall Accuracy: 95.06  
Sensitivity: 0.8829431030234219  
Specificity: 0.879431030234219  
Precision: 0.8870981485377656  
F1 Score: 0.8750155778312503
```

Fig 6.2.3: Model Performance Evaluation on Yassen Dataset Using Log-mel Spectrogram signal

```

*** Overall Accuracy: 91%
Sensitivity: 0.8529431030234219
Specificity: 0.8529431030234219
Precision: 0.8570981485377656
F1 Score: 0.8550155778312503
SNR: 9.445405613222372
MCC: 0.710029094133016
Kappa: 0.7099945994263732

```

Overall Accuracy: 91%

UAR: 85.29

Fig 6.2.4: Model Performance Evaluation on PhysioNet Dataset Using Linear Spectrogram

80	conv2d_13 1024 1×1×1024 convolutions with stride ...	2-D Convolution	$3(S) \times 2(S) \times 1024(C) \times 1(B)$	Wei... $1 \times 1 \times 1024$ Bias $1 \times 1 \times 1024$	-
81	l132 Batch normalization with 1024 channels	Batch Normalization	$3(S) \times 2(S) \times 1024(C) \times 1(B)$	Offs... $1 \times 1 \times 1024$ Scale $1 \times 1 \times 1024$	TrainedMean $1 \times 1 \times 1024$ TrainedVariance $1 \times 1 \times 1024$
82	activation_26 ReLU	ReLU	$3(S) \times 2(S) \times 1024(C) \times 1(B)$	-	-
83	global_average_pooling2d 2-D global average pooling	2-D Global Average ...	$1(S) \times 1(S) \times 1024(C) \times 1(B)$	-	-
84	dense 5 fully connected layer	Fully Connected	$1(S) \times 1(S) \times 5(C) \times 1(B)$	Weights 5×1024 Bias 5×1	-
85	softmax softmax	Softmax	$1(S) \times 1(S) \times 5(C) \times 1(B)$	-	-
86	Sounds crossentropyx with 'class1' and 4 other ...	Classification Output	$1(S) \times 1(S) \times 5(C) \times 1(B)$	-	-

Fig 6.2.5 Final layer of the modified YamNet

```

Duration: 21.872
Loaded: /content/drive/MyDrive/MINIPROJECT/physionet_split/physionet_split/test/abnormal/e01526.wav
Duration: 9.941
Train shape: (29836, 96, 64, 1)
Test shape: (7569, 96, 64, 1)

```

Fig 6.2.6 Fragment Extracted

```

indices = [0, 100, 200, 300, 400]

for i in indices:
    pred = np.argmax(net.predict(X_test[i:i+1]))
    print(f"Index {i} → Pred: {pred} | Actual: {y_test[i]}")

1/1 ----- 0s 139ms/step
Index 0 → Pred: 0 | Actual: 0
1/1 ----- 0s 210ms/step
Index 100 → Pred: 2 | Actual: 1
1/1 ----- 0s 151ms/step
Index 200 → Pred: 2 | Actual: 2
1/1 ----- 0s 172ms/step
Index 300 → Pred: 3 | Actual: 3
1/1 ----- 0s 225ms/step
Index 400 → Pred: 4 | Actual: 4

```

Fig 6.2.7 Prediction made by Model trained with Yaseen Dataset (Spectrogram)